1. The initial audio recording has an extensive 1D wave structure. However, keyword recognition becomes easier after translating the signal into a time-frequency graph that demonstrates the transformation of energy at various frequencies within small frames. These spectrogram/MFCC-type features enable easy learning of speech characteristics by the model and lower the processing load of the network.

2. The representative dataset refers to a few actual input samples that are used in the full integer quantization process. This is important since TensorFlow Lite makes use of the input samples to determine the appropriate scaling and zero point for the INT8 quantizer. This would ensure that the quantized model can retain its accuracy when executed in the embedded devices. In the notebook, the representative dataset is derived from the audio_processor.get_data() function and passed to the converter from the representativedatasetgen.

3. audio_processor.get_data() comes from input_data.py and returns prepared audio feature data and labels for a chosen split such as training, validation, testing, or representative calibration data. It handles dataset partitioning and preprocessing so the model receives the same kind of feature input it was trained on. In the notebook, it is used both for evaluation and for generating representative samples for quantization.

4. g_model refers to the C/C++ byte array generated by the TensorFlow Lite model, while g_model_len is the size of the byte array. Both values need to be copied exactly into the Arduino code because they are used directly as the TensorFlow Lite Micro model because if either is incorrect, the model will fail to load and run on the board.

5.